

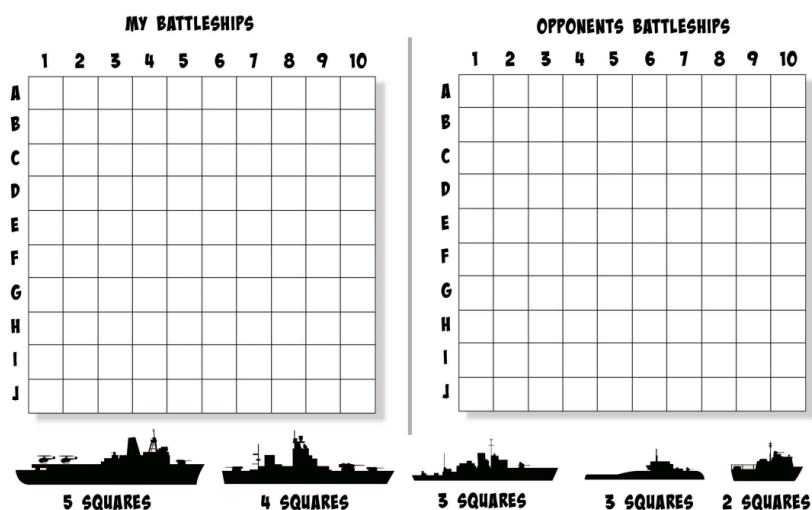
Programming Fundamentals (Term Project)

Introduction

Battleship™ is a popular classic strategy-type guessing game for two-players. It is played on ruler grids of 10x10 dimensions (for each player) on which each player's fleet of warships are marked and set up before the battle begins. The location of the fleets are concealed from the other player. Players alternate turns calling "shots" at the other player's ships, and the objective of the game is to destroy the opposing player's fleet. The more moves a player takes, the lesser the score they will have. The game ends when either of the player's fleet is completely destroyed.

Game Rules

Each player has two grids, one for placing their ships and another for recording their guesses of the opponent's grid. The grids are generally labeled with letters for rows and digits for columns. Each player has a fleet of ships of varying sizes that they place on the grid. Ships can only be placed horizontally or vertically, not diagonally. Ships cannot overlap, they must fit within the grid boundaries. The fleet consists of 5 ship types that may be placed over the board: 1 Aircraft Carrier (5 Spaces), 1 Battleship (4 Spaces), 1 Submarine (3 Spaces), 1 Cruiser (3 Spaces) and 1 Destroyer (2 Spaces). A general template to conceptualize game components is as follows:



The goal is to sink all of the opponent's ships by correctly guessing their locations on the grid. Players take turns guessing a grid coordinate (e.g., B5) on the opponent's board. If a player's guess hits a ship, the opponent announces a hit, and the attacker marks that coordinate as hit, else the opponent announces a miss and the attacker marks it as a miss on the board. The coordinate that has already been shot at, cannot be shot again. When all coordinates of the ship have been hit, the opponent announces the ship as sunk. The attacker's goal is to identify and sink all of the opponent's ships. The first player to sink all of the opponent's ships wins the game.

This project introduces a slight addition to the game. All the classical game rules apply, however, the game would be played by a single user, while the other player is simulated by the computer. The computer generates its share of fleet design at random every time and shoots its shots at random as well, while ensuring that no game rules are violated.

Meanwhile, the scoring system is as follows: the initial score is always set to 100 (since there are 100 possible maximum moves due to $10 \times 10 = 100$ total spaces) and the score is decremented after every shot/move, i.e. the lesser the moves a player takes, higher their score is. After a user wins, both of the players reveal their boards; the user name is then prompted and saved for display in the scoreboard (which displays the top three player highscores).

Project Guidelines

- Implementation of a graphical user interface is **mandatory**. You may use any graphic library of your choice but SFML is recommended. Outstanding user interfaces may be awarded bonus points.
- Plagiarism of any kind would strictly not be tolerated and result in a **zero**.
- The game should be implemented fully functional with all the game rules.
- The game should have four navigation screens: Main Menu Screen (for New Game, Leaderboard, etc.), Fleet Formation Screen (for the user to place their ships), Battle Screen (for the battle) and Leaderboard Screen (for displaying top 3 players and their scores). You may include more screens (such as splash/loading screens, pop-up screen for prompting user name at the end at etcetera). You have the creative freedom to incorporate your own ideas as well, for example introducing an optional player vs player mode as well additionally if you want to. Bonus for extra features.
- The game should allow the user to view the leaderboard, which should be achieved and implemented via appropriate file handling.
- The computer-simulated player should be implemented through random generation for fleet formation and shot coordinate decision via the functions provided in C++. These too should abide by the game rules and not violate them.
- For the computer-simulated short coordination decision, if a shot successfully hits a user battleship, then the subsequent shots must not be random anymore. The computer might try horizontally or vertically to completely destroy the user ship simulating an AI-like attempt at destroying ships. Once the found user ship is destroyed, it gets back to shooting shots at random again.
- The game boards should be implemented through 2D-arrays and appropriately updated. Proper memory allocation and deallocation should be done.

Helpful Links

- Sample Battleship gameplay for reference: [Sample Battleship Gameplay](#)
- SFML for Visual Studio Guide: <https://www.sfml-dev.org/tutorials/2.6/start-vc.php>
- Official SFML Documentation: <https://www.sfml-dev.org/tutorials/2.6/>
- SFML Playlist: [SFML Usage & Setup Playlist](#)
- Another SFML Playlist: [Another SFML Usage & Setup Playlist](#)
- SFML Sprite Movement Video: [SFML Sprite Movement Video](#)